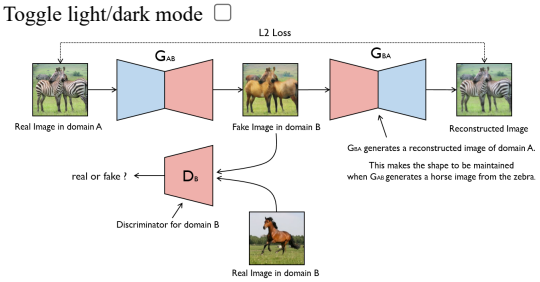
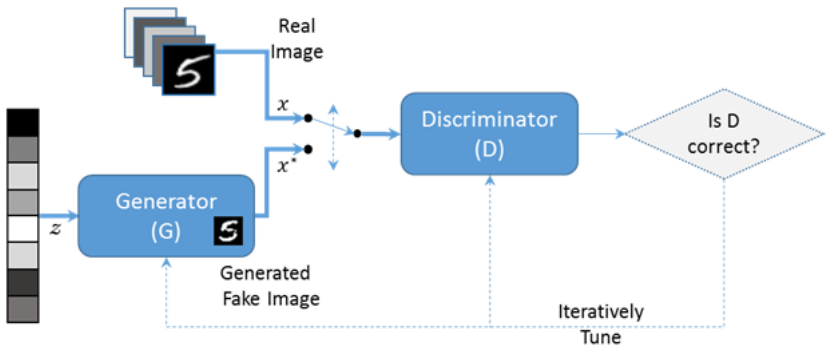


Train your first CycleGAN for Image to Image Translation

Artificial-Intelligence • Mar 31, 2021

General Adversarial Networks or GAN, were first introduced in a paper by **Ian Goodfellow** and other researchers outlining a new deep neural network architecture comprising of two networks. The two networks, the ***Generator (G)*** and ***Discriminator (D)*** are in a competitive game of min max. The Generator generates new classes of data while the Discriminator tries to evaluate the authenticity of this new data. The ultimate goal is for the Generator to blur the lines between generated and authentic data, and thus succeed in fooling the Discriminator. In the architecture below, GAN is trying to generate images of number 5 from MNIST dataset.



GAN Architecture

In this tutorial, we will train a **CycleGAN** model to translate photos of horses to zebras, and back again to horses. We will be training the model for 20 epoc… Kaggle using GPU and compare the performance of GAN. The architecture used in the tutorial will comprise of 2 discriminator & generator models each and this is

the very model which was trained in the CycleGAN paper.

For the scope of this tutorial, we will keep the helper functions out, however, they will be available in the notebook at the end of the tutorial.

Why CycleGAN?

While there has been a great deal of research into this task, most of it has utilised supervised training, where we have access to (x, y) pairs of corresponding images from the two domains we want to learn to translate between. CycleGAN is interesting because it did not require paired training data.

> Paired data is harder to find in most domains, and not even possible in some, the unsupervised training capabilities of CycleGAN are quite useful.

Let's get into the tutorial.

Step 1: Dataset Loading and Preprocessing

Load the Dataset and extract images to convert to an array of images which will be stored in a compresses NumPy array format. We will also need helper functions to load & generate real samples and to generate fake samples.

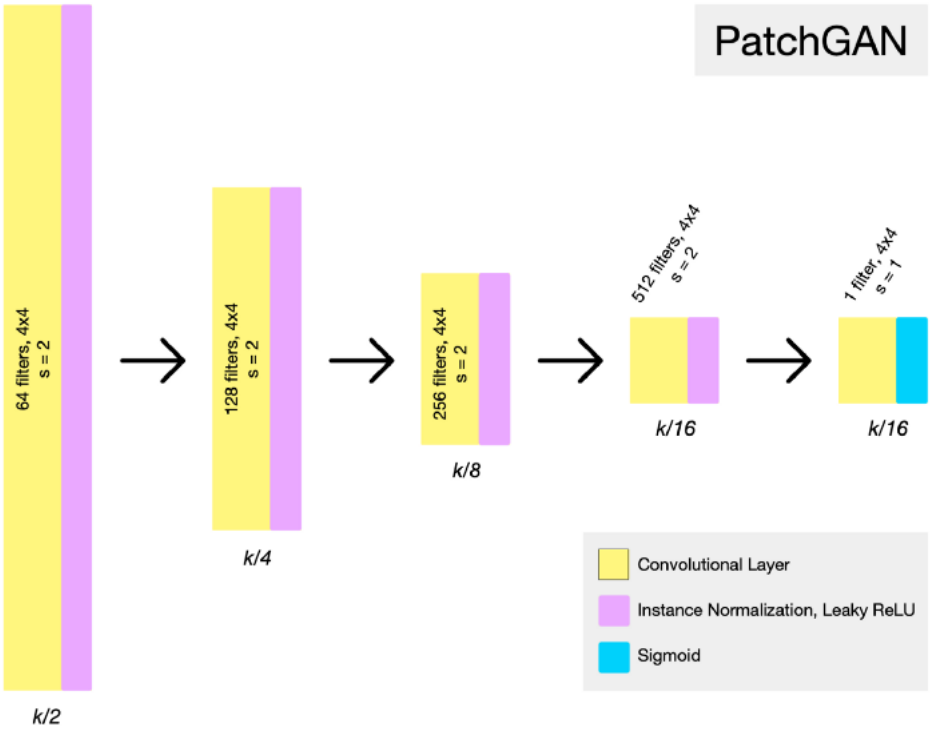
```
from os import listdir
from numpy import asarray
from numpy import vstack
from keras.preprocessing.image import img_to_array
from keras.preprocessing.image import load_img
from numpy import savez_compressed

# load all images in a directory into memory
def load_images(path, size=(256,256)):
    data_list = list()
    # enumerate filenames in directory, assume all are images
    for filename in listdir(path):
        # load and resize the image
        pixels = load_img(path + filename, target_size=size)
        # convert to numpy array
        pixels = img_to_array(pixels)
        # store
        data_list.append(pixels)
    return asarray(data_list)

# dataset path
path = '../input/horse-to-zebra/horse2zebra/'
# load dataset A
dataA1 = load_images(path + 'trainA/')
dataAB = load_images(path + 'testA/')
dataA = vstack((dataA1, dataAB))
print('Loaded dataA: ', dataA.shape)
# load dataset B
dataB1 = load_images(path + 'trainB/')
dataB2 = load_images(path + 'testB/')
dataB = vstack((dataB1, dataB2))
print('Loaded dataB: ', dataB.shape)
# save as compressed numpy array
filename = './horse2zebra_256.npz'
savez_compressed(filename, dataA, dataB)
print('Saved dataset: ', filename)
```

Step 2: Making the Discriminator (D) model

The Discriminator is a deep convolutional neural network that performs image classification. It takes a source image as input and predicts the likelihood of whether the target image is a real or fake image. Two discriminator models are used, one for Domain-A (horses) and one for Domain-B (zebras).



PatchGAN

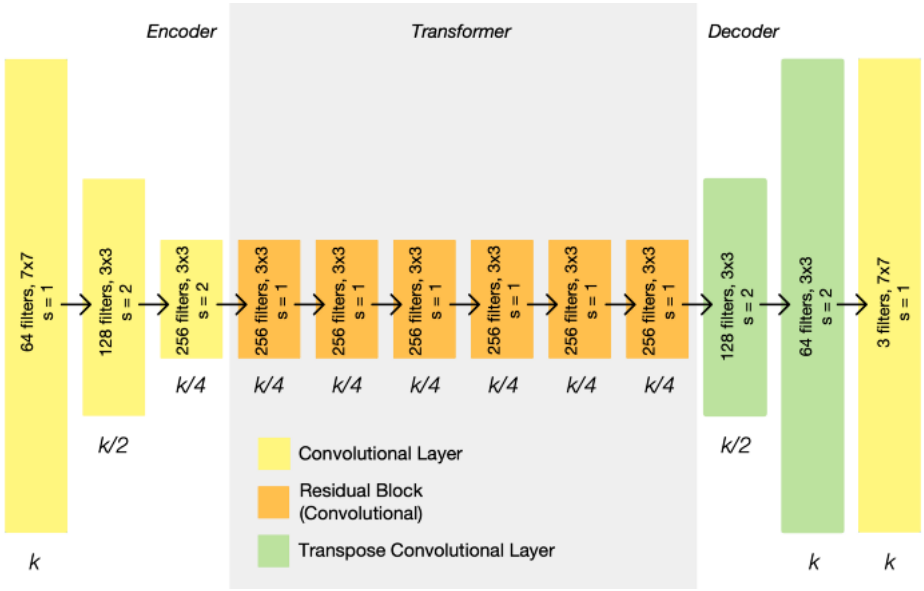
The discriminator design is based on the effective receptive field of the model, which defines the relationship between one output of the model to the number of pixels in the input image. This is called a PatchGAN model and is carefully designed so that each output prediction of the model maps to a 70×70 square or patch of the input image. The benefit of this approach is that the same model can be applied to input images of different sizes, e.g. larger or smaller than 256×256 pixels.

```
def define_discriminator(image_shape):
    # weight initialization
    init = RandomNormal(stddev=0.02)
    # source image input
    in_image = Input(shape=image_shape)
    # C64
    d = Conv2D(64, (4,4), strides=(2,2), padding='same', kernel_initializer=init)(in_image)
    d = LeakyReLU(alpha=0.2)(d)
    # C128
    d = Conv2D(128, (4,4), strides=(2,2), padding='same', kernel_initializer=init)(d)
    d = InstanceNormalization(axis=-1)(d)
    d = LeakyReLU(alpha=0.2)(d)
    # C256
    d = Conv2D(256, (4,4), strides=(2,2), padding='same', kernel_initializer=init)(d)
    d = InstanceNormalization(axis=-1)(d)
    d = LeakyReLU(alpha=0.2)(d)
    # C512
    d = Conv2D(512, (4,4), strides=(2,2), padding='same', kernel_initializer=init)(d)
    d = InstanceNormalization(axis=-1)(d)
    d = LeakyReLU(alpha=0.2)(d)
    # second last output layer
    d = Conv2D(512, (4,4), padding='same', kernel_initializer=init)(d)
    d = InstanceNormalization(axis=-1)(d)
    d = LeakyReLU(alpha=0.2)(d)
    # patch output
    patch_out = Conv2D(1, (4,4), padding='same', kernel_initializer=init)(d)
    # define model
    model = Model(in_image, patch_out)
    # compile model
    model.compile(loss='mse', optimizer=Adam(lr=0.0002, beta_1=0.5), loss_weights=[0.5])
    return model
```

Step 3: Making the Generator (G) Model

The generator is an encoder-decoder model architecture. The model takes a source image (e.g. horse photo) and generates a target image (e.g. zebra photo) this by first downsampling or encoding the input image down to a bottleneck layer, then interpreting the encoding with a number of ResNet layers that use

connections, followed by a series of layers that upsample or decode the representation to the size of the output image. So, we first define a ResNet block and then our Generator Model.



CycleGAN Generator

```
def resnet_block(n_filters, input_layer):
    # weight initialization
    init = RandomNormal(stddev=0.02)
    # first layer convolutional layer
    g = Conv2D(n_filters, (3,3), padding='same', kernel_initializer=init)(input_layer)
    g = InstanceNormalization(axis=-1)(g)
    g = Activation('relu')(g)
    # second convolutional layer
    g = Conv2D(n_filters, (3,3), padding='same', kernel_initializer=init)(g)
    g = InstanceNormalization(axis=-1)(g)
    # concatenate merge channel-wise with input layer
    g = Concatenate()([g, input_layer])
    return g


def define_generator(image_shape, n_resnet=9):
    # weight initialization
    init = RandomNormal(stddev=0.02)
    # image input
    in_image = Input(shape=image_shape)
    # c7s1-64
    g = Conv2D(64, (7,7), padding='same', kernel_initializer=init)(in_image)
    g = InstanceNormalization(axis=-1)(g)
    g = Activation('relu')(g)
    # d128
    g = Conv2D(128, (3,3), strides=(2,2), padding='same', kernel_initializer=init)(g)
    g = InstanceNormalization(axis=-1)(g)
    g = Activation('relu')(g)
    # d256
    g = Conv2D(256, (3,3), strides=(2,2), padding='same', kernel_initializer=init)(g)
    g = InstanceNormalization(axis=-1)(g)
    g = Activation('relu')(g)
    # R256
    for _ in range(n_resnet):
        g = resnet_block(256, g)
    # u128
    g = Conv2DTranspose(128, (3,3), strides=(2,2), padding='same', kernel_initializer=init)(g)
    g = InstanceNormalization(axis=-1)(g)
    g = Activation('relu')(g)
    # u64
    g = Conv2DTranspose(64, (3,3), strides=(2,2), padding='same', kernel_initializer=init)(g)
    g = InstanceNormalization(axis=-1)(g)
```

```
    g = Activation('relu')(g)
    # c7s1-3
    g = Conv2D(3, (7,7), padding='same', kernel_initializer=init)(g)
    g = InstanceNormalization(axis=-1)(g)
    out_image = Activation('tanh')(g)
    # define model
    model = Model(in_image, out_image)
    return model
```

The discriminator models are trained directly on real and generated images, whereas the generator models are not.

Step 4: Combine the models

```
def define_composite_model(g_model_1, d_model, g_model_2, image_shape):
    # ensure the model we're updating is trainable
    g_model_1.trainable = True
    # mark discriminator as not trainable
    d_model.trainable = False
    # mark other generator model as not trainable
    g_model_2.trainable = False
    # discriminator element
    input_gen = Input(shape=image_shape)
    gen1_out = g_model_1(input_gen)
    output_d = d_model(gen1_out)
    # identity element
    input_id = Input(shape=image_shape)
    output_id = g_model_1(input_id)
    # forward cycle
    output_f = g_model_2(gen1_out)
    # backward cycle
    gen2_out = g_model_2(input_id)
    output_b = g_model_1(gen2_out)
    # define model graph
    model = Model([input_gen, input_id], [output_d, output_id, output_f, output_b])
    # define optimization algorithm configuration
    opt = Adam(lr=0.0002, beta_1=0.5)
    # compile model with weighting of least squares loss and L1 loss
    model.compile(loss=['mse', 'mae', 'mae', 'mae'], loss_weights=[1, 5, 10, 10], optimizer=opt)
    return model
```

Step 5: Train the model

We will need helper functions in order to save Models at checkpoints, summarise Performance during training and update Image Pool for fake images.

```
def train(d_model_A, d_model_B, g_model_AtoB, g_model_BtoA, c_model_AtoB, c_model_BtoA, dataset):
    # define properties of the training run
    n_epochs, n_batch, = 20, 1
    # determine the output square shape of the discriminator
    n_patch = d_model_A.output_shape[1]
    # unpack dataset
    trainA, trainB = dataset
    # prepare image pool for fakes
    poolA, poolB = list(), list()
    # calculate the number of batches per training epoch
    bat_per_epo = int(len(trainA) / n_batch)
    # calculate the number of training iterations
    n_steps = bat_per_epo * n_epochs
    # manually enumerate epochs
    for i in range(n_steps):
        # select a batch of real samples
        X_realA, y_realA = generate_real_samples(trainA, n_batch, n_patch)
        X_realB, y_realB = generate_real_samples(trainB, n_batch, n_patch)
        # generate a batch of fake samples
        X_fakeA, y_fakeA = generate_fake_samples(g_model_BtoA, X_realB, n_patch)
```

```
            X_fakeB, y_fakeB = generate_fake_samples(g_model_AtoB, X_realA, n_patch)
            # update fakes from pool
            X_fakeA = update_image_pool(poolA, X_fakeA)
            X_fakeB = update_image_pool(poolB, X_fakeB)
            # update generator B->A via adversarial and cycle loss
            g_loss2, _, _, _, _  = c_model_BtoA.train_on_batch([X_realB, X_realA], [y_realA, X_realA, X_realB, X_r
            # update discriminator for A -> [real/fake]
            dA_loss1 = d_model_A.train_on_batch(X_realA, y_realA)
            dA_loss2 = d_model_A.train_on_batch(X_fakeA, y_fakeA)
            # update generator A->B via adversarial and cycle loss
            g_loss1, _, _, _, _ = c_model_AtoB.train_on_batch([X_realA, X_realB], [y_realB, X_realB, X_realA, X_re
            # update discriminator for B -> [real/fake]
            dB_loss1 = d_model_B.train_on_batch(X_realB, y_realB)
            dB_loss2 = d_model_B.train_on_batch(X_fakeB, y_fakeB)
            # summarize performance
            print('>%d, dA[%.3f,%.3f] dB[%.3f,%.3f] g[%.3f,%.3f]' % (i+1, dA_loss1,dA_loss2, dB_loss1,dB_loss2, g_
            # evaluate the model performance every so often
            if (i+1) % (bat_per_epo * 1) == 0:
                # plot A->B translation
                summarize_performance(i, g_model_AtoB, trainA, 'AtoB')
                # plot B->A translation
                summarize_performance(i, g_model_BtoA, trainB, 'BtoA')
            if (i+1) % (bat_per_epo * 5) == 0:
                # save the models
                save_models(i, g_model_AtoB, g_model_BtoA)
```

Train Function

```
# load image data
dataset = load_real_samples('./horse2zebra_256.npz')
print('Loaded', dataset[0].shape, dataset[1].shape)
# define input shape based on the loaded dataset
image_shape = dataset[0].shape[1:]
# generator: A -> B
g_model_AtoB = define_generator(image_shape)
# generator: B -> A
g_model_BtoA = define_generator(image_shape)
# discriminator: A -> [real/fake]
d_model_A = define_discriminator(image_shape)
# discriminator: B -> [real/fake]
d_model_B = define_discriminator(image_shape)
# composite: A -> B -> [real/fake, A]
c_model_AtoB = define_composite_model(g_model_AtoB, d_model_B, g_model_BtoA, image_shape)
# composite: B -> A -> [real/fake, B]
c_model_BtoA = define_composite_model(g_model_BtoA, d_model_A, g_model_AtoB, image_shape)
# train models
train(d_model_A, d_model_B, g_model_AtoB, g_model_BtoA, c_model_AtoB, c_model_BtoA, dataset)
```

The above training process for 20 epochs took around 8 hours with GPU accelerator on Kaggle and post training, the converted images looked a little like:

Converting Zebras back to Horses

It's clear the performance is not that good as of now, but after training it for around 60 epochs, it really starts to shine. Sadly, it can't be trained on Kaggle for 60 epochs because of the 9-hour limit but if you have a PC with a decent GPU, you can choose to train this locally.

[Optional]

Step 6: Use the saved models in order to Generate Images:

```
def select_sample(dataset, n_samples):
    # choose random instances
    ix = randint(0, dataset.shape[0], n_samples)
    # retrieve selected images
    X = dataset[ix]
    return X

# plot the image, the translation, and the reconstruction
def show_plot(imagesX, imagesY1, imagesY2):
    images = vstack((imagesX, imagesY1, imagesY2))
    titles = ['Real', 'Generated', 'Reconstructed']
    # scale from [-1,1] to [0,1]
    images = (images + 1) / 2.0
    # plot images row by row
    for i in range(len(images)):
        # define subplot
        pyplot.subplot(1, len(images), 1 + i)
        # turn off axis
        pyplot.axis('off')
        # plot raw pixel data
        pyplot.imshow(images[i])
        # title
        pyplot.title(titles[i])
    pyplot.show()

# load dataset
A_data, B_data = load_real_samples('./horse2zebra_256.npz')
print('Loaded', A_data.shape, B_data.shape)
# load the models
cust = {'InstanceNormalization': InstanceNormalization}
model_AtoB = load_model('./g_model_AtoB_023740.h5', cust)
model_BtoA = load_model('./g_model_BtoA_023740.h5', cust)
# plot A->B->A
A_real = select_sample(A_data, 1)
B_generated  = model_AtoB.predict(A_real)
A_reconstructed = model_BtoA.predict(B_generated)
show_plot(A_real, B_generated, A_reconstructed)
# plot B->A->B
B_real = select_sample(B_data, 1)
A_generated  = model_BtoA.predict(B_real)
B_reconstructed = model_AtoB.predict(A_generated)
show_plot(B_real, A_generated, B_reconstructed)
```

CycleGAN - Image to Image Translation
Explore and run machine learning code with Kaggle Notebooks | Using data from [Private Datasource]

Kagglerazor08

[Bonus]

Understand and apply GANs using the following tutorial from Jason Brownlee of MachineLearningMastery.

A Gentle Introduction to Generative Adversarial Networks (GANs)
Generative Adversarial Networks, or GANs for short, are an approach to generative modeling using deep learning methods, such as convolutional ne networks. Generative modeling is an unsupervised learning task in machine learning that involves automatically discovering and learning the regularitie…

Machine Learning MasteryJason Brownlee

Cheers!

P.S. - I have been wondering about changing the frequency of posting due to constraints, if I choose to do that, you can read about it here.

Tags

- Artificial-Intelligence
- GAN
- CycleGAN
- Generator
- Discriminator
- Blog
- Classifiers
- Deep-Learning
- Image
- Machine-Learning
- Translation
- Tutorial

Subscribe to our newsletter

Get the latest posts delivered right to your inbox.

Your email address Your email address
Subscribe

Now check your inbox and click the link to confirm your subscription.

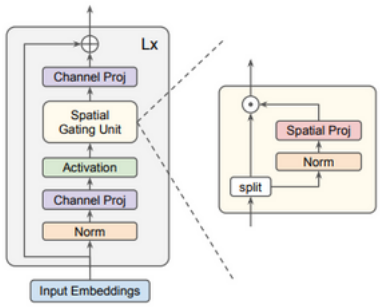
Please enter a valid email address

Oops! There was an error sending the email, please try later.

Jay Sinha

-
-

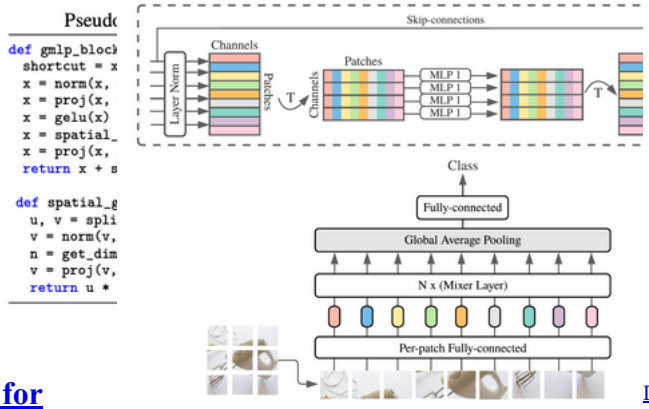
Recommended for you



Computer Vision

Train your first gMLP Model for classifying images

a year ago • 5 min read



Computer Vision

Train your first MLPMixer model for classifying images

a year ago • 6 min read

Deep-Learning

Train your first Neural Ne Optical Character Recogni

2 years ago • 7 min read